

Rebel Profiler - GUI Build Brief (paste this whole file into ChatGPT)

How to use: Paste this entire document into ChatGPT (or any capable AI) and ask it to build the frontend. It contains everything: product context, the exact backend API, every screen's spec, and the visual design system that matches the Hermes agent's terminal aesthetic.

0. Your role (AI, read this first)

You are a senior full-stack engineer building a local-first security dashboard GUI for an existing, working backend. The backend is DONE and tested (776 tests, CI green) - you only build the frontend that talks to it. The look and feel must match the backend's agent personality: "Hermes" - a dark, terminal-inspired, hacker-operator aesthetic that still feels modern, clean and beautiful. Think: Kali Linux meets a modern SaaS dashboard.

1. Product in one paragraph

Rebel Profiler is a case-based, local-first security recon framework. The operator creates a *case*, authorizes *scope* (targets they are allowed to touch), activates the case, then collects data via tools (DNS, whois, cert-transparency, web crawling, search-engine dorking, the operator's own browser via an extension). Everything becomes hash-chained evidence and claims (structured facts with confidence scores). A surface graph links hosts/IPs/technologies, and a final report is generated. The AI agent Hermes (a local 7B GGUF model via llama.cpp) drives everything through plain-language chat and tool calls. NOTHING leaves the machine.

2. The Hermes visual identity (this is the vibe)

The CLI banner looks like this - the GUI should feel like its descendant:

```

      _-(_)-  | \
    _-( _)-  | \
  _-(_)- _-( | \
 _-( _)- _-( | \
-( _)- _-( _ | ;
  - - - - - | -'
      \ | \
      \ |
      \ |
                                Model  dphn/Qwen2.5-Coder-7B...
                                Case    394c8b4dd5a1 [ACTIVE]
                                Tools   33 operator + 16 adapters
REBEL PROFILER - HERMES
authorized security operations /help
the LLM proposes - the gates decide

                                hermes?
```

- * Tone: operator-console. Dense but readable. Monospace accents.
- * Copy style: short, honest, a little swagger ("the LLM proposes - the gates decide"). Status glyphs: ok x (o) <> (green/red/cyan).

3. Design system (use exactly this)

```
/* Kali-dark palette */
--bg-0:      #0b0e14; /* app background */
--bg-1:      #11151f; /* panels/cards */
--bg-2:      #1a2030; /* inputs, hover surfaces */
--border:    #232b3d; /* hairlines
```

```
--text:      #d7dce6; /* primary text */
--text-dim:   #7d8698; /* secondary text */
--accent:     #00e5a0; /* hermes green (primary CTA, live states) */
--accent-2:   #38bdf8; /* cyan (links, info, focus) */
--warn:       #fbbf24; /* amber (approvals, risk) */
--danger:     #f43f5e; /* red (denied, out-of-scope) */
--ok:         #22c55e; /* success */

/* Type */
--font-ui:    "Inter", system-ui, sans-serif; /* body */
--font-mono:  "JetBrains Mono", "Fira Code", monospace; /* data, banner, claims */
```

Rules: dark theme only. Panels have 1px --border, 12px radius, 16px padding. Data (claims, hashes, IPs, evidence ids) ALWAYS monospace. Accent green only for actions/live states, never for decoration. Every status has a glyph + color + text (never color alone). Subtle scanline/grid background texture on the shell background is welcome; keep it very low-contrast.

4. Backend surface (real, tested, use exactly this)

4a. Read-only HTTP API - `rebel-profiler serve <case-id> --port 8899 --token SECRET`

- * Auth: Authorization: Bearer <sha256_hex(SECRET)> on every request.
- * GET /healthz -> {"ok": true}
- * GET / -> {"service": "rebel-profiler api", "version": 1, "case_id": "..."}
 - * GET /state -> {"case_id": <int>, "claims": <int>, "evidence": <int>, "audit_head": "<hash or null>"}
 - * GET /events -> {"events": [...recent 50 structured events...]}
- * Errors: 401 (bad token), 405 (non-GET), 404. Read-only - there is NO write API over HTTP.

4b. Everything else is CLI, always callable with `-o json`

The GUI should have a small Python (FastAPI) or Node proxy that shells out to these and returns JSON - never parse human text:

```
rebel-profiler case list -o jsonl      # all cases [{id,name,status,...}]
rebel-profiler case show <id> -o json  # case detail
rebel-profiler case create "<name>" "<desc>" -o json
rebel-profiler case scope add <id> <value> -o json
rebel-profiler case activate <id> -o json
rebel-profiler intel claims <id> -o json # all claims (kind,value,confidence,evidence_id,...)
rebel-profiler report <id> -o json      # findings + evidence links
rebel-profiler surface map <id> -o json  # graph nodes + edges
rebel-profiler audit verify <id> -o json # chain_ok true/false
rebel-profiler adapters -o json         # available tool adapters
rebel-profiler intel collect <id> <action> <target> -p k v -y -o json
rebel-profiler intel collect <id> dork-search <domain> -p engine google|duckduckgo|ahmia -p dork <
rebel-profiler browser submit <id> <url> --extract title,links,text -o json
rebel-profiler browser result <job-id> -o json
```

Dork names: site-files, open-directories, config-files, backup-files, login-portals, staging-sites, cloud-buckets, exposed-emails, tech-stack.
Engines: google, duckduckgo, ahmia (Tor; needs tld=onion and a bare name target).

4c. Hermes chat

Interactive CLI REPL. For the GUI, run it as a child process and stream

stdout over SSE/WebSocket through the proxy:

```
rebel-profiler hermes --tier high --case <id> [-o json]
```

Output lines include the banner, hermes? prompt echo, hermes is working..., tool-call lines [t1] ok claims_list, the hermes> answer and a (N turn(s), Xs) footer. Parse loosely; ALWAYS show raw lines too (a "terminal view" pane). Important honesty rule: the model is local and slow (first turn can take minutes on CPU) - the UI must say so and show a live elapsed timer, never a fake progress bar.

5. Screens to build

S1 - Shell / App frame

Left icon rail (Dashboard, Cases, Hermes, Dorks, Surface, Bridge, Settings).
Top bar: active case chip (<> 394c8b4dd5a1 - ACTIVE in mono+green), audit chain state (ok/x), bridge status dot. Background: --bg-0 with faint grid.

S2 - Dashboard

- * Stat tiles: Claims, Evidence, Audit head (short hash, click = copy), Open findings (from report), each with a sparkline if events allow.
- * Live events feed (GET /events polled 5s): severity-colored rows, mono timestamps, filter chips.
- * Quick actions: "Ask Hermes", "New case", "Run dork", "Grab URL".

S3 - Cases

- * Card/table list: id (mono), name, status pill (draft=amber, active=green, closed=dim). "New case" wizard modal: name -> description -> scope entries (repeatable input, wildcards allowed like *.lab.test) -> activate toggle with a big honest warning ("scope enforcement goes live").

S4 - Case detail (the workhorse)

Tabs: Overview - Claims - Evidence - Report - Surface.

- * Overview: case meta, scope entries table (include/exclude), activate button (if draft), counts.
- * Claims: dense mono table (id, subject, kind, value, confidence bar, evidence link, state). Filter by kind + text search. Click row -> drawer with full provenance (source, method, observed_at, task id).
- * Evidence: list with hash (mono, truncated, click = full + copy), kind, size, source. Verify-all button -> audit verify banner result.
- * Report: findings cards grouped, confidence, evidence links, "generated at".
- * Surface: graph view of surface map nodes/edges (use Cytoscape.js or D3-force; dark theme, nodes = host/ip/tech with distinct shapes, edges labeled with relation like resolves_to). Fallback: simple SVG tree.

S5 - Hermes chat (flagship screen - make it beautiful)

Layout: chat column + right sidebar (case context: claims count, tools list).

- * Banner header reproducing the ASCII style (render in mono, green accents).
- * Message list: user bubbles right (you>), hermes answers left (hermes>), tool calls as compact timeline chips [t1] ok dns-lookup example.test

(ok green / x red / spinner while running), turn footer (2 turn(s), 41s).

- * A terminal view toggle: raw REPL output in a mono pane (autoscroll).

- * Input: hermes? prompt style, Enter sends, slash-command autocomplete (/help /status /tools /scope /clear).

- * While working: "hermes is working..." with a live elapsed timer and the honest note "first turn reads the whole tool contract - can take minutes on CPU". NEVER a fake progress bar.

- * Suggested prompts row: "map this domain", "what's exposed?", "generate the report", "grab the login page".

S6 - Dorking console

- * Select target (from scope), engine (3 radio cards: Google / DuckDuckGo / Ahmia-over-Tor with a "requires torsocks" badge), dork (dropdown with descriptions), then Run -> POST via proxy to intel collect, stream the JSON result: evidence id, claims table as they parse, empty-state "engine returned no hits (that's data too)".

S7 - Browser bridge

- * Status card: bridge up/down (poll 8765 /healthz through proxy), case, scope count, token display (masked, click to reveal/copy).

- * "Grab a URL" form: url + extractor checkboxes (title/text/links/headers/forms/cookies/meta) -> browser submit -> job id chip with live state -> result drawer with data preview (title, links list, text excerpt).

- * Setup help panel: embed the 5 Firefox steps from docs/EXTENSION_SETUP.md.

S8 - Settings/Health

- * API connection form (port, secret) stored in localStorage, big green ok when /healthz + /state both OK. Show backend version from GET /.

- * Diagnostics list: audit chain, bridge, tor (for ahmia), model presence.

6. Proxy (thin, provide it)

Provide proxy.py (FastAPI, ~150 lines) or proxy.mjs (Express):

- * GET /api/state|events|healthz -> forwards to 127.0.0.1:8899 with the computed bearer token (secret from env/CLI arg, never sent to browser).

- * POST /api/cli body {cmd: ["case", "list", "-o", "json"], } -> runs the whitelisted rebel-profiler binary ONLY with -o json* appended, 120s timeout, returns stdout parsed as JSON. Whitelist subcommands; never interpolate shell strings (use spawn with argv array).

- * GET /api/hermes/stream (SSE) -> spawns hermes --tier high --case <id> -o json child, streams stdout lines as events, accepts a prompt via query param on first connect or a POST that writes to child stdin.

7. Stack + project layout (frontend/ folder)

```
frontend/  
  proxy.py          # or proxy.mjs  
  src/  
    main.ts(x) / App.svelte  
    api.ts          # typed fetch wrappers (state, events, cli, hermes)  
    theme.css       # the design tokens from §3
```

components/	# Panel, StatTile, ClaimTable, Glyph, StatusPill, ...
screens/	# Dashboard, Cases, CaseDetail, Hermes, Dorks, Bridge
index.html	

Any stack is fine (React+Vite, Svelte, or even Alpine+htmx) - but deliver dark theme + mono data discipline + the screens above. TypeScript preferred. No cloud calls; everything binds 127.0.0.1.

8. Interaction law (non-negotiable)

1. Honest progress: real timers + real states only; never fake bars.
2. Fail-closed visuals: out-of-scope/denied states are loud (red, glyph, plain-language reason + fix hint from the backend error action field).
3. Evidence everywhere: every claim/finding links its evidence id.
4. Mono for data, UI font for prose - never blur the two.
5. Local-only: all requests to 127.0.0.1; show the target of every network call in Settings.

9. Acceptance checklist

- * [] Dashboard tiles live-update from /state + /events
- * [] Case create -> scope add -> activate completes without terminal
- * [] Claims table renders 100+ rows fast, filters by kind/text
- * [] Surface graph draws nodes/edges from surface map JSON
- * [] Hermes chat streams a real run, shows tool-call timeline + terminal view
- * [] Dork console runs a real dork-search and renders claims
- * [] Bridge screen shows live status and completes one rp grab job
- * [] Audit verify shows ok chain OK banner
- * [] Everything works offline, dark theme, zero console errors

10. Real sample data (for your mockups)

```
{ "case_id": "394c8b4dd5a1", "claims": 7, "evidence": 5,
  "audit_head": "51eeaf6fdcb83a58" }
// claims sample
{ "id": "cl_d3742fa78c20", "subject": "www.hplovecraft .com", "kind": "ip",
  "value": "38.247.139.179", "confidence": 1.0, "source": "dns.authoritative",
  "method": "dns-lookup", "state": "open" }
{ "id": "cl_d9acab6b88df", "subject": "www.hplovecraft .com", "kind": "web_finding",
  "value": "header:CSP:missing", "confidence": 0.5, "source": "scan.web",
  "method": "web-audit", "state": "open" }
// hermes reply sample
"hermes> The web findings for www.hplovecraft.com include missing Content
Security Policy, HSTS, X-Content-Type-Options and X-Frame-Options headers."
```

Now build it. Ask yourself for any detail not covered here, then produce the complete project: files, code, and a run/README. Make it gorgeous.